Career Decision Support System — Project Write-Up

Project Write-Up

Career Decision Support System CS 4580/5580 Final Project — Kevin Rusagara Iraguha (individual project)

What it is

A program that recommends careers from a user's profile and explains why. The user enters their technical skills, non-technical skills, interests, work-style preferences, goals, constraints, and prior experience. The system runs them against a hand-authored rule base of 64 rules covering 13 career paths, then produces a ranked recommendation with full explanations.

The point isn't the recommendation — recommenders are easy. The point is that **every score is fully traceable**. You can ask "why did you put PM above consulting?" and the system answers in concrete terms: "PM gets +1.5 from your interest in users; consulting gets +3.0 from your interest in strategy but loses -2.0 because of your no_relocation constraint; their delta is +1.5 in PM's favor."

How to run it

The fastest way to see it work:

```
python3 -m advisor                  # interactive — prompts for your
        inputs
python3 -m advisor --preset swe     # run a sample profile
python3 -m advisor --list           # list 9 sample profiles
```

For the full notebook walkthrough (sample profiles + interactive ipywidgets form):

```
jupyter notebook career_advisor.ipynb
```

CLI and engine use only the Python standard library, so the Zoo runs it with no setup. The notebook needs `jupyter` and `ipywidgets`, both standard in the Zoo's Jupyter environment.

Techniques used

The project handout listed several suggested techniques. I used three of them:

1. **Rule-based expert system** ("Build a rule base expert system" in the handout). 64 rules grouped into 7 categories: skills, interests, work style, experience, goals, constraints, and compound rules. Each rule declares a condition (a callable that inspects the profile) and a weighted-effect dict (which careers it pushes up or down, by how much). A small DSL — `any_skill`, `all_skills`, `lacks_skill`, `has_interest`, `has_work_style`, `has_constraint`, `has_experience_in`, `goal_mentions`, `either` — keeps the rule definitions readable.
2. **Option generation** ("Option generation: if a or b, why not c?" in the handout). When the top two careers are within `close_threshold` of each other, the system surfaces a curated **hybrid role** for the pair — e.g., SWE + Data Science → "ML Engineering"; SWE + PM → "Technical Product Manager". It also flags **dark-horse careers** (high positive evidence held back by penalties) and prints a **weak-match warning** when even the top option scores below `weak_threshold`.
3. **Qualitative arithmetic / heat-map summary** ("Qualitative arithmetic — when is a number a 'good number' or a 'bad number'?" and the equity- portfolio example *"summary heat map analysis (Green, Yellow, Red)"*). I collapse each numeric career score into a **GREEN / YELLOW / RED** confidence band — GREEN ≥ 10, YELLOW ≥ 5, RED < 5 — and annotate every recommendation in `explain_top` inline with its band. The full `heat_map(scores)` view groups all 13 careers by band. An all-RED heat map is the system saying *"your profile doesn't yet give enough signal for any of these careers"* — a single qualitative summary the raw numbers don't communicate.

How decisions are explained

Every score the system produces can be traced back step by step:

- A **score** is the sum of contributions from all rules that fired.
- A **contribution** is `rule.effects[career_id] × match.strength`.
- A **match** records the strength (0..1) and the `evidence` — the human-readable strings naming the exact profile attributes that triggered the rule (e.g., `"python: 5/5"`, `"interest: ai"`, `"goal mentions 'start a company'"`).
- A **rule** has an ID and a description that's surfaced verbatim in the explanation output.

There's no opaque weight matrix, no embedding, no learned representation. If a user disagrees with a recommendation, they can read the trace and identify the specific rule they disagree with — and the system tells them what would have to change for the recommendation to flip.

The explanation engine has six surfaces:

Surface	What it answers
`explain_top`	"Why did you recommend this?" — top positive and negative factors per career
`head_to_head`	"Why this one over that one?" — the rules that distinguish two careers

Surface	What it answers
`counterfactuals`	"What would change your mind?" — minimal profile perturbations that would flip the top recommendation
`detect_tradeoffs`	"What's pulling against itself?" — rules that simultaneously favor one career and penalize another
`heat_map`	"How confident are you?" — qualitative GREEN/YELLOW/RED summary
`alternatives`	"What other options should I consider?" — hybrid roles, dark horses, weak-match warnings

What's interesting about it

Three concrete demonstrations from the sample profiles:

1. **Constraint flips the recommendation.** The `consulting_candidate` profile has strong communication, an explicit consulting goal, *and* a `no_relocation` constraint. The system ranks Product Management above Consulting because the constraint penalizes consulting (jobs cluster in major cities). The counterfactual then says: *"If the constraint 'no_relocation' were removed, top recommendation would shift to Management Consulting."* A black-box recommender would hide this — here the user can see the constraint is the only thing blocking their stated preference.
2. **Compound rules find non-obvious fits.** A compound rule like `ml_plus_systems` fires only when the user has both `machine_learning ≥ 3` and `system_design ≥ 3`. For a profile with strong SWE skills + ML basics + systems depth, this lifts ML Engineering above straight SWE. An attribute-by-attribute scorer would miss the synthesis.
3. **Honest weak-match warnings.** The `early_career_candidate` profile is intentionally underspecified. The system refuses to fake confidence — it prints a `WEAK MATCH` warning saying *"even the top option scores only +2.00."* All 13 careers land in the RED band of the heat map, and the counterfactuals double as career guidance: here are the specific skills that, if developed, would shift the picture.

Implementation

- **Language:** Python 3, stdlib only for the engine and CLI. Notebook uses `jupyter` and `ipywidgets`.
- **Engine:** ~1,000 lines across `advisor/profile.py`, `advisor/careers.py`, `advisor/rules.py`, `advisor/scoring.py`, `advisor/explanation.py`, `advisor/alternatives.py`.
- **CLI:** `advisor/__main__.py` — interactive prompts plus `--preset` / `--list` flags.
- **Sample profiles:** 9 profiles in `sample_profiles/profiles.py`, designed to exercise different decision scenarios (clean fits, robust fits, constraint-driven demotions, conflicts, weak signals).
- **Tests:** `tests/test_engine.py` — 27 tests run by `python3 -m unittest tests.test_engine` in under 10 ms. Cover structural invariants (no rules with impossible conditions, ≥45% sample coverage), per-profile sanity rankings, all six

explanation surfaces, alternative-engine triggers, counterfactual robustness, heat-map band thresholds, score reproducibility, and CLI smoke tests.

Files

```
career_advisor.ipynb        notebook walkthrough (executed, outputs
included)
advisor/
  __init__.py
  __main__.py               CLI entry point
  profile.py                UserProfile dataclass
  careers.py                13-career catalog
  rules.py                  64 rules + DSL helpers
  scoring.py                rule-firing + score aggregation
  explanation.py            6 explanation modes
  alternatives.py           option generation
sample_profiles/
  profiles.py               9 sample profiles
tests/
  test_engine.py            27 tests
docs/screenshots/           6 screenshots of the notebook
README.md                   short index
PROJECT.md                  this write-up
requirements.txt
```